Sprig 2 Language Design Document

Programming language design and implementation • 9 November 2026

1 Problem Statement

1.1 Purpose and intended users

Sprig 2 is a small, interpreted language for students learning how state, control flow, functions, and scope work. Its design makes declaration, branching, repetition, and function boundaries visible in ordinary text. A single Python source file contains its lexer, parser, abstract syntax tree, environments, and evaluator [1]. The goal is a language that students can both use and explain, including the path from a source statement to its runtime effect.

The primary users are beginning programmers moving from pseudocode or visual blocks to textual programs. Their tasks include computing totals, classifying a score, repeating a calculation, and decomposing an algorithm into functions. A secondary user is a programming-languages student or instructor who needs a compact implementation for tracing tokens, AST nodes, lexical lookup, and error propagation. The intended setting is a classroom exercise, command-line demonstration, or small laboratory assignment.

1.2 The problem to solve

A beginner must often learn an algorithm and the surrounding language conventions at the same time. A condition may accept values other than true and false; an operator may convert values implicitly; a visually small indentation change may alter a block. These are legitimate design choices, but they add rules that a learner must track before being able to predict an elementary program. Sprig addresses this limited teaching problem by making conditions boolean-only, separating declaration from assignment, and closing every compound statement with end.

The problem is not that established languages cannot express elementary algorithms. They can, and they offer far broader libraries and tooling. Sprig instead chooses a small semantic surface for exercises whose purpose is to understand execution. Its educational value is a design hypothesis; correct implementation alone cannot establish that students learn faster or make fewer mistakes.

1.3 Concrete sources of surprise

Python accepts nonempty strings as true and treats bool as a numeric subtype [2]. A student intending to store a logical flag may therefore write the following program and reach the branch even though the text says “false”. Sprig accepts a string value but rejects using it as a condition, requiring the learner to make the intended boolean explicit.

```
# Python
ready = "false"
if ready:
    print("run")        # run
print(True + 1)          # 2

# Sprig
let ready = "false"
when ready:
    print("run")
end                     # Line 3: Expected a boolean (true or false)
```

Python also uses indentation to group suites [3]. Sprig retains indentation for readability while using a colon, newline, and end as the actual delimiters. This costs additional lines, but a learner can identify where a nested block finishes without interpreting whitespace as syntax. Neither choice eliminates the need to understand nesting.

1.4 Survey of existing languages and tools

The survey compares four relevant approaches. These observations concern documented language behavior and interface choices, rather than measured teaching outcomes. Each system offers a different balance between explicit syntax, expressiveness, and the amount of machinery exposed to the learner.

Language or tool	How it handles the problem	Implication for Sprig
Python	Readable statements and indentation-based suites; conditions use truth-value testing [2], [3].	Retain direct expressions and functions; require explicit boolean values and end delimiters.
JavaScript	The + operator supports numeric addition and string concatenation, with coercion when needed [4].	Reject mixed string/number addition so the meaning of + follows the operand types.
C	Operators have defined precedence and associativity; arithmetic, relational, logical, and assignment operators form a compact expression model [5].	Keep a small precedence table and explicit runtime checks instead of adopting C conversion and pointer rules.

For a concrete coercion example, JavaScript evaluates "5" + 1 to the string "51" [4]. In Sprig, print("5" + 1) raises a type error; print("5" + "1") produces 51 and print(5 + 1) produces 6. The distinction forces the program to state whether it is joining text or adding numbers. Sprig has no conversion built-ins, so this consistency also limits convenience.

For fixed repetition, a Python learner can write for i in range(3), while Sprig offers repeat 3. The latter avoids introducing an iteration variable when the exercise only needs three executions. A while loop remains available for termination controlled by changing state. Scratch supplies a useful precedent for exposing a repetition count directly [5]; Sprig does not reproduce Scratch's visual editor or event model.

1.5 Scope and success criteria

The implementation includes numbers, strings, booleans, nil, variables, arithmetic, comparisons, selection, two loops, functions, recursion, closures, output, and an interactive prompt. It excludes collections, classes, modules, file access in the language, graphical events, and concurrency. These boundaries keep the interpreter inspectable and the specification short enough to teach as a complete system.

Criterion	Acceptance evidence
C1 Core expressiveness	Run arithmetic, selection, both loops, recursion, and function-value examples with exact expected output.
C2 Predictable values	Reject boolean arithmetic, nonboolean conditions, invalid repeat counts, and mixed string/number addition; verify short-circuit evaluation.
C3 Consistent binding	Verify local shadowing, nearest-binding assignment, lexical closure capture, and retained closure state.
C4 Actionable failure	Reject malformed syntax before execution; report line-numbered runtime errors and a nonzero file-mode exit status.
C5 Accessible execution	Run from Python 3.10 or newer without third-party packages; support file execution, AST inspection, and REPL recovery.
C6 Teaching suitability	Have learners predict outputs and repair representative mistakes. A learner study is future validation, not a completed result.

Section 5 evaluates C1–C5 using reproducible implementation checks. C6 remains unmeasured. The intended contribution is the coherent combination of established ideas in a small teaching language, not a claim that any individual construct is new.

2 Design Specification

2.1 Personas and use cases

Mika, a novice learner, computes a score and prints a pass/fail result using let and when. Andre, a student tracing algorithms, compares repeat with while and implements factorial recursively. An instructor inspects --ast output, demonstrates a closure, and asks students to explain a line-numbered error. These are design personas, not participants in a user study. Each maps directly to the programs in Section 2.6.

2.2 Complete concrete syntax

The following EBNF specifies Sprig 2 [1]. Braces mean zero or more repetitions, brackets mean optional content, and quoted text is a terminal. EOF denotes end of input. Spaces, tabs, and comments are discarded; newlines remain significant. A final source statement need not end with a newline because the lexer supplies one.

```
program       = { NEWLINE | line }, [ statement ], EOF ;
line          = statement, NEWLINE ;
body          = NEWLINE, { NEWLINE | line } ;

statement     = declaration | assignment | function | return
              | selection | while | repeat | expression ;
declaration   = "let", NAME, "=", expression ;
assignment    = NAME, "=", expression ;
function      = "fn", NAME, "(", [ parameters ], ")", ":", body, "end" ;
parameters    = NAME, { ",", NAME } ;
return        = "return", [ expression ] ;
selection     = "when", expression, ":", body,
                [ "otherwise", ":", body ], "end" ;
while         = "while", expression, ":", body, "end" ;
repeat        = "repeat", expression, ":", body, "end" ;

expression    = disjunction ;
disjunction   = conjunction, { "or", conjunction } ;
conjunction   = negation, { "and", negation } ;
negation      = "not", negation | comparison ;
comparison    = sum, [ comparison-op, sum ] ;
comparison-op = "==" | "!=" | "<" | "<=" | ">" | ">=" ;
sum           = product, { ( "+" | "-" ), product } ;
product       = unary, { ( "*" | "/" | "%" ), unary } ;
unary         = "-", unary | call ;
call          = primary, { "(", [ arguments ], ")" } ;
arguments     = expression, { ",", expression } ;
primary       = NUMBER | STRING | "true" | "false" | "nil" | NAME
              | "(", expression, ")" ;

NUMBER        = digit, { digit }, [ ".", digit, { digit } ] ;
NAME          = letter, { letter | digit } ;
letter        = "a" ... "z" | "A" ... "Z" | "_" ;
digit         = "0" ... "9" ;
STRING        = '"', { string-char | escape }, '"' ;
escape        = '\', ( '"' | '\' | "n" | "t" | "r" ) ;
string-char   = ? any character except double quote, backslash, CR, or LF ? ;
NEWLINE       = LF | CR, [ LF ] ;
comment       = "#", { ? any character except CR or LF ? } ;
space         = " " | TAB ;
LF            = ? U+000A line feed ? ;
CR            = ? U+000D carriage return ? ;
TAB           = ? U+0009 tab ? ;
```

Reserved words: let, fn, return, when, otherwise, end, while, repeat, true, false, nil, and, or, not. Names are case-sensitive ASCII identifiers; a keyword cannot be a name. Decimal literals require digits on both sides of the decimal point. No exponent notation, single quotes, multiline strings, or multiline expressions are accepted.

Context constraints: return must occur in a function body and parameter names must be distinct. Comparisons do not chain; use a < b and b < c. Assignment targets must be bare names. Blank lines and empty bodies are legal. Each compound statement has its own end.

2.3 Abstract syntax and execution model

The parser produces a list of Node(kind, line, args) records. Each node retains its source line for diagnostics. The AST removes punctuation and grouping parentheses; precedence is represented by the tree structure. In the table below, E is an expression node, B a list of statement nodes, and names and operators are strings. These are the complete node variants emitted by the parser [1].

Node kind	Ordered args fields	Meaning
literal	(value,)	Number, string, boolean, or nil
name	(name,)	Resolve the nearest lexical binding
unary	(operator, E)	Numeric negation or boolean not
binary	(operator, left, right)	Arithmetic, comparison, or boolean operator
call	(callee, arguments)	Call an evaluated function value
let	(name, E)	Declare a new name in the current environment
assign	(name, E)	Replace the nearest existing binding
expression	(E,)	Evaluate an expression as a statement
fn	(name, parameters, B)	Declare a closure in the current environment
return	(E,)	Return a value; bare return uses literal nil
when	(condition, yes B, no B)	Choose one branch; absent otherwise is []
while	(condition, B, [])	Repeat while the condition is true
repeat	(count, B, [])	Execute B a fixed number of times

2.4 Semantics of the constructs

Execution proceeds through the statement list in source order. A literal evaluates to its stored value; a name retrieves a binding. Unary minus negates a number, while not negates a boolean. Binary arithmetic evaluates left then right. Calls bind more tightly than unary minus, followed by multiplication/division/remainder, addition/subtraction, comparison, not, and, and or. Arithmetic operators of equal precedence associate left. Parentheses override grouping.

The and and or operators evaluate the left operand first and require a boolean. False and skips its right operand; true or skips its right operand. Otherwise the right operand is evaluated and must also be boolean. All other binary operators evaluate both operands. Equality is numeric across integer/float values, false across other differing types, value-based for strings, booleans, and nil, and identity-based for functions.

A let statement evaluates its initializer and then declares the name locally; duplicate local declarations fail. Assignment evaluates its right side and updates the nearest existing binding. Neither statement returns a value. An expression statement evaluates its expression; file execution does not automatically print the result. The REPL echoes a non-nil final result, so print is necessary for explicit file output.

A when statement executes exactly one branch after checking its boolean condition. A while statement checks before every iteration, including the first. A repeat statement evaluates the count once, requires a non-negative integer, and executes its body exactly that many times unless a return or error exits it. Blocks reuse the surrounding environment; a repeated let can therefore fail on a later iteration.

A function declaration stores Function(params, body, closure), retaining the defining environment by reference. A call evaluates the callee and then every argument left to right, checks callability and exact user-function arity, and creates a local environment whose parent is the captured environment. Return exits the nearest active function even from a nested branch or loop. Falling off the body or a bare return yields nil. The built-in print accepts any number of values, joins their display forms with spaces, appends a newline, and returns nil.

2.5 Types scope and error handling

Sprig is dynamically typed: values have runtime types, while a variable may later hold a different type. The value domain is Number (integer or finite float), String, Boolean, Nil, and Function. There are no type annotations, inference pass, implicit string/number conversions, or static type-safety proof. Runtime checks reject invalid operations when they are reached; unexecuted branches can still contain runtime type errors.

Operation	Required values and outcome
+, -, *, /, %, unary -	Numbers; + also accepts two strings. / uses true division; % follows floor-division remainder. Zero divisors fail.
<, <=, >, >=	Two numbers; result is boolean. Booleans are excluded from numeric operations.
==, !=	Any two values; numeric 1 and 1.0 compare equal, but true and 1 do not. Functions compare by identity.
when, while, not, and, or	Booleans only; no truthiness conversion. Logical operators short-circuit.
repeat and calls	repeat accepts an integer >= 0, excluding 3.0 and true. User functions require exact arity; a nonfunction cannot be called.

Binding is lexical and resolved at runtime. The environment chain is the call-local environment, its defining environments, and the built-in environment. Function calls create scopes; when, while, and repeat do not. A local declaration may shadow an outer name. Assignment searches outward and changes the first match. A closure retains a live environment, so later assignments are visible; it does not capture a frozen copy. A function is available after its declaration executes, allowing recursion through its own environment. There is no declaration hoisting. The built-in print can be shadowed or reassigned.

Core operational rules

Let ρ be an environment chain, σ a store mapping binding locations to values, and O the output sequence. Write ⟨e,ρ,σ,O⟩ ⇓ ⟨v,σ′,O′⟩ for expression evaluation and ⟨s,ρ,σ,O⟩ ⇒ ⟨ρ′,σ′,O′,k⟩ for a statement, where k is normal or return(v). lookup finds the nearest location. extend creates a child environment. The following big-step rules specify a core subset; errors propagate instead of producing a successful judgment.

```
Literal:  <v, rho, sigma, O> evaluates to <v, sigma, O>.
Name:     lookup(rho,x)=l => x evaluates to sigma(l).
Let:      evaluate e to v; require x absent locally; allocate fresh l;
          bind x to l in rho and set sigma(l)=v; completion is normal.
Assign:   evaluate e to v; lookup(rho,x)=l; set sigma(l)=v.
AndFalse: evaluate e1 to false => e1 and e2 evaluates to false;
          keep e1's store/output and do not evaluate e2.
When:     evaluate c to boolean b; execute B_yes if b, else B_no.
While:    evaluate c; false finishes; true executes B then repeats,
          threading the store/output unless B returns or fails.
Call:     evaluate callee to <params,B,rho_c>, then actuals in order;
          check arity; execute B in extend(rho_c,params=actuals).
          return(v) yields v; normal completion yields nil.
```

These rules model mutable captured bindings through shared store locations. Sequence passes each updated state to the next statement; return(v) propagates through sequences and loops until the enclosing call consumes it. Function declaration inserts a closure referencing the same environment in which its name is installed.

Failure and recovery

Lexing and parsing finish before execution, so a syntax error prevents all statements in that submission from running. Runtime errors stop at the failing operation; earlier assignments and printed output remain. Errors include undefined names, duplicate declarations, wrong types, arity mismatch, invalid counts, zero division, and excessive numeric results or recursion. SprigError reports a line number; file mode adds the filename and exits with status 1. There is no language-level try/catch. The REPL clears the failed input buffer and preserves its environment; :reset creates a new environment.

2.6 Example programs

Each program is independent. Output is exact, with one newline per printed line. All ten were executed against the documented interpreter.

1 Basic output

```
print("Hello, Sprig!")
```

Output

```
Hello, Sprig!
```

2 Variables and assignment

```
let score = 10
score = score + 5
print("Score:", score)
```

Output

```
Score: 15
```

3 Arithmetic expressions

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(5 / 2)
print(7 % 3)
```

Output

```
14
20
2.5
1
```

4 Strings

```
let first = "Hello"
let second = "World"
print(first + " " + second)
```

Output

```
Hello World
```

5 Boolean logic and comparisons

```
let ready = true
let battery = true
print(ready and battery)
print(ready or false)
print(not ready)
print(10 > 5)
```

Output

```
true
true
false
true
```

6 Conditional statements

```
let score = 85
when score >= 75:
    print("Passed")
otherwise:
    print("Try again")
end
```

Output

```
Passed
```

7 While loop

```
let count = 1
while count <= 3:
    print(count)
    count = count + 1
end
```

Output

```
1
2
3
```

8 Functions and return values

```
fn double(n):
    return n * 2
end

print(double(5))
```

Output

```
10
```

9 Repeat loop

```
repeat 3:
    print("Welcome to Sprig")
end
```

Output

```
Welcome to Sprig
Welcome to Sprig
Welcome to Sprig
```

10 Recursion

```
fn factorial(n):
    when n <= 1:
        return 1
    end

    return n * factorial(n - 1)
end

print(factorial(5))
```

Output

```
120
```

3 Paradigm Synthesis

3.1 Imperative and structured programming

Sprig combines an imperative foundation with functional programming features. The imperative foundation makes execution order and changing state explicit. A program creates a binding with let, changes it with assignment, selects a path with when, and repeats statements with while or repeat. The accumulation example in Section 2.6 is imperative because its total depends on a sequence of updates. Reordering those updates can change the result.

Structured control flow organizes this stateful execution into nested blocks, each with a visible entry and end. There is no goto, and no break or continue. Return is a controlled early exit from a function. Procedural programming describes how named functions organize reusable work within this model; it is not counted as a separate third paradigm merely to increase the paradigm count.

3.2 Functional influence

Functions are first-class values: a function can be bound to another name, passed to a function, returned as a result, or retained after its defining call has ended. Example 7 passes inc to twice, which applies the supplied operation twice. The caller chooses the behavior without duplicating the control structure. This is higher-order programming, rather than simply calling a fixed named procedure.

Closures pair a function body with access to its defining environment. Example 8 reads the outer x even when the caller has a local x with a different value. Example 9 returns next from counter and retains the binding of n across later calls. Racket provides an established model of functions as values and explicit binding regions [6], [7]; Sprig uses similar underlying ideas while choosing a different textual syntax and a much smaller feature set.

Sprig has no anonymous lambda syntax, but this does not prevent first-class use: fn first binds a function value, after which that value is passed or returned normally. Recursive factorial expresses a computation in terms of a smaller call. Recursion and higher-order calls offer alternatives to loops, although there is no tail-call optimization and recursive depth is limited by the host runtime.

3.3 How the paradigms combine

The two paradigms share one evaluator and one value model. A call expression may appear on an assignment's right side, inside a condition, or as an argument to another call. A function may perform imperative work and still be returned as a value. No special bridge or conversion is needed because a closure is already a value in the interpreter. The environment chain connects this value-level behavior to the store used by assignment.

A useful classroom progression is to write an accumulator using a loop, extract a calculation into a function, pass that function to another function, and finally return a closure that retains state. Each step adds a distinct idea while reusing the same expression and call rules. The AST makes the progression visible: binary and call nodes appear inside the same statement forms, instead of requiring separate execution engines.

3.4 Tensions and consequences

Mutable closures are the central tension. A pure function's result depends only on its inputs, but the next closure in Example 9 returns different values on consecutive calls without arguments. It is not referentially transparent, and replacing repeated calls with a previously computed value changes behavior. Sprig permits this to demonstrate encapsulated state; it does not claim to be a purely functional language.

Evaluation order therefore matters. Sprig evaluates the callee first, then arguments left to right, so print(c(), c()) produces 1 2. Short-circuit operators may intentionally skip effects on the right. Capturing a live environment supports useful shared state but also makes closures sensitive to later assignments. Teaching these effects requires an environment-and-store explanation, not only a substitution model.

Dynamic typing and lexical scope are language properties, not additional paradigms. Python classes used to implement the interpreter do not make the Sprig language object-oriented. Likewise, a REPL reads input interactively but does not provide event-driven language constructs. Keeping these distinctions explicit makes the paradigm claim match the implemented language.

4 Design Rationale

Four decisions explain the current design. The appendix records them as a retrospective decision log; this section connects their benefits and costs to programming-language theory. Alternatives describe design choices considered in this analysis, not a fabricated history of dated meetings.

4.1 Explicit blocks and two repetition forms

Decision D1 is to use newline-delimited statements and colon/end blocks, with when/otherwise, while, and repeat. Alternatives are indentation-sensitive suites, brace-delimited blocks, or a single general loop. Python's indentation-based suites demonstrate one compact alternative [3]. Sprig's delimiters make block boundaries explicit while keeping whitespace formatting conventional. They also make the grammar and parser straightforward to trace.

The trade-off is verbosity: every nested block adds an end, and a missing end remains a syntax error. The second loop is deliberate duplication at the semantic level. While expresses repetition governed by a boolean condition; repeat expresses a count evaluated once. Keeping both gives students a direct contrast between condition-controlled and counted iteration. Structured control flow constrains the possible execution paths, but does not guarantee that a while loop terminates.

4.2 Dynamic values with strict operator contracts

Decision D2 is dynamic typing with explicit runtime checks. Alternatives are static annotations and a type-checking pass, or permissive coercion and truthiness. Static checking could catch some errors before execution, but would require additional syntax and type rules beyond the small interpreter. Permissive conversion is convenient, as illustrated by JavaScript addition [4], but can obscure whether an operation is numeric or textual.

Sprig therefore permits a name to change value type while requiring an operation to receive suitable operands. True is not a number, conditions accept booleans only, and + accepts numbers or two strings. This is a runtime discipline, not a formal proof of type safety. A wrong type in an unexecuted branch is not detected, and equality intentionally accepts all value pairs. The benefit is a small rule set students can test; the cost is later error discovery and the absence of conversion utilities.

4.3 Lexical closures with function level scope

Decision D3 combines lexical binding, first-class closures, and scopes introduced only by function calls. Dynamic scope was rejected because a free variable would depend on the caller's active environment rather than the function's defining environment. Capturing values by snapshot would avoid some mutation effects but would not support the shared counter semantics. Lexical binding instead makes the defining environment the basis for explaining a name [7].

Block scope was another alternative. Sprig shares branch and loop environments to keep the model small: a let in a selected branch remains available afterward, while a let in an untaken branch never executes. A let inside a repeated loop can fail on its second execution. This behavior is consistent but requires teaching, and it is a candidate for future revision. Nearest-binding assignment allows outer mutation without a separate nonlocal form. It also makes accidental state changes possible, so closure examples must show which environment owns each name.

4.4 A transparent interpreter and narrow feature set

Decision D4 is a dedicated lexer, recursive-descent statement parser with precedence-based expression parsing, and tree-walking evaluator. Alternatives include translating source into Python, evaluating host-language text, or building bytecode and a virtual machine. The selected pipeline makes every step inspectable through source code and --ast, and Sprig source is never sent to Python eval or exec [1].

The trade-off is execution speed and host-runtime dependence. User recursion consumes Python call frames, and values reuse Python integers, floats, and strings under Sprig's own guards. Restricting the language to one output built-in and omitting collections, inheritance, and modules limits application scope. It also keeps composition through ordinary function calls and higher-order functions central: reuse comes from passing and combining behavior rather than from class hierarchies.

These decisions optimize for a coherent teaching artifact. They should be revisited if the target shifts toward production scripting, large data sets, static verification, or graphical applications; those goals would justify a different scope and implementation budget.

5 Evaluation

5.1 Method and observed results

Evaluation used the current working-tree version of sprig.py, grammar.ebnf, and tests/test_sprig.py on 9 November 2026 [1]. The command python3 -m unittest discover -s tests -v passed all 13 test methods, including subtests for 12 successful example files and four expected-error files. The ten programs in Section 2.6 were also run independently and matched their stated outputs. These counts describe test methods and examples, not code coverage or a user-study sample.

Criterion	Finding and supporting check
C1 Supported	Arithmetic and precedence, nested selection/loops, factorial recursion, and ten documented examples produced expected results.
C2 Supported	Type rejection and short-circuit checks passed; repeat count is evaluated once. An additional probe confirmed that 3.0 is rejected.
C3 Supported	Tests verified lexical capture, closure mutation, local shadowing, and persistent environments; Example 8 distinguishes lexical from caller-based lookup.
C4 Supported	Syntax errors precede execution; runtime failures include line information. CLI checks verify status 1 and no Python traceback for a representative error.
C5 Supported locally	CLI, AST output, REPL blocks, reset, echo, and recovery checks passed. Cross-platform compatibility was not separately measured.
C6 Not measured	No classroom experiment or usability study was conducted; learning and debugging benefits remain hypotheses.

5.2 Comparison with Python on the same tasks

Equivalent Sprig and Python programs were executed for the tasks below. Counts include nonblank, noncomment source lines and explicit closing end lines. Both versions produced identical outputs on these tasks. Line count describes surface cost, not readability or performance.

Task	Sprig lines	Python lines	Shared output
Classify score 85 against threshold 75	6	5	Passed
Add one three times using a counted loop	5	4	3
Compute factorial of 5 recursively	7	5	120

Sprig is longer in these cases because blocks end explicitly. Python already performs all three tasks concisely; Sprig's distinction is the restricted set of rules exposed during the exercise. On boundary cases, Python accepts a nonempty string as a condition and True + 1 as arithmetic [2], whereas Sprig reports errors. This comparison supports semantic differences, not a claim that Sprig outperforms Python or improves learning.

5.3 Limitations and future work

The implementation has no collections, indexing, for, break, continue, classes, modules, user input primitive, or exception handling. Expressions occupy one line. Recursion and numeric behavior depend partly on the Python host, and there is no execution budget for nonterminating loops. Diagnostics generally identify a line but do not show columns, call stacks, or multiple errors. Runtime failure does not roll back earlier state changes. Shared block scope and mutable capture require careful explanation.

The next priorities are a grammar-based regression suite, clearer error locations, tests at numeric and recursion limits, and a documented decision on block-local bindings. A learner evaluation should compare matched tasks in Sprig and Python, counterbalance order, record completion time and error categories, and ask students to predict scope and short-circuit behavior. Those measurements would test C6. Adding features should follow evidence that they serve the teaching goal rather than simply narrowing the gap with general-purpose languages.

References

[1] “Sprig,” GitHub repository. Accessed: Nov. 9, 2026. [Online]. Available: https://github.com/htoisenglar-1/Sprig

[2] Python Software Foundation, “Built-in types,” Python 3.12 documentation. Accessed: Nov. 10, 2026. [Online]. Available: https://docs.python.org/3.12/library/stdtypes.html

[3] Python Software Foundation, “Compound statements,” Python 3.12 language reference. Accessed: Nov. 10, 2026. [Online]. Available: https://docs.python.org/3.12/reference/compound_stmts.html

[4] MDN contributors, “Addition (+),” MDN Web Docs. Accessed: Nov. 10, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Addition

[5] cppreference.com contributors, “C operator precedence,” C language reference. Accessed: Nov. 10, 2026. [Online]. Available: https://en.cppreference.com/w/c/language/operator_precedence

Appendix

A Design decision log

Recorded 9 November 2026 as a retrospective description of the implemented design. This recording date does not assert when the original choices were made. D1–D4 correspond to Section 4.

ID	Selected design	Alternative and consequence
D1	Colon/end blocks; while and repeat	Indentation, braces, or one loop: shorter or more uniform syntax, but a different teaching model for boundaries and counts.
D2	Dynamic values; strict runtime contracts	Static typing adds a checker; coercion adds implicit conversions. Current errors occur when operations execute.
D3	Lexical closures; function scopes	Dynamic scope or capture by copy changes free-variable behavior. Block scope could avoid repeated-let failures.
D4	Dedicated tree-walking interpreter	Host translation or bytecode could change complexity/performance. Current design prioritizes inspection and a small implementation.

B Contribution record

Member	Contribution
Labergue	Design document Sections 1 to 3: Problem Statement, Design Specification (syntax, type system, 10 example programs), Paradigm Synthesis
Gallo	Design document Section 4 (Design Rationale), Section 5 (Evaluation), and this Appendix (contributions and decision log)
Lar	Implementation (interpreter, tests, README), User guide, and Presentation